



eSim Semester Long Internship Spring 2026

on

OpenROAD Integration with eSim

Submitted by

Rishabh Jain

Jaypee University of Engineering & Technology

Under the guidance of

Prof. Prabhu Ramachandran

Principal Investigator

Department of Aerospace Engineering

Indian Institute of Technology Bombay

May 30, 2026

Acknowledgement

I would like to express my sincere gratitude to Prof. Prabhu Ramachandran for providing me the opportunity to be a part of the FOSSEE internship program and for supporting open-source engineering tool development at IIT Bombay. His guidance and vision have encouraged students and researchers to actively contribute to the open-source ecosystem.

I would also like to acknowledge Prof. Kannan M. Moudgalya for his foundational role in establishing and nurturing the FOSSEE initiative. His contributions toward open-source education and the creation of the FOSSEE fellowship framework have directly shaped the platform through which this internship was undertaken.

My sincere appreciation extends to my mentor, Mr. Sumanto Kar, for his continual support, technical guidance, and encouragement throughout the duration of this project. His insights and feedback played a key role in refining ideas, overcoming challenges, and ensuring the successful completion of the assigned tasks.

I would also like to thank my internal mentors, Mr. Varad Patil and Ms. Shanthi Priya K, for their valuable guidance, coordination, and technical inputs during the internship. Their mentorship contributed significantly to the clarity, progress, and successful execution of the work.

This internship has been an enriching learning experience, allowing me to work closely with open-source EDA tools, integrate OpenROAD into eSim, and gain practical exposure to RTL-to-GDSII digital design flows. The knowledge acquired during this period will undoubtedly support my future academic and professional pursuits.

I would also like to thank the entire FOSSEE team and the open-source community for their coordination, assistance, and timely interactions at various stages of this work. Their collective efforts ensured smooth workflow, resource accessibility, and effective project execution.

Rishabh Jain

Abstract

eSim is a free and open-source Electronic Design Automation (EDA) tool developed by the FOSSEE team at IIT Bombay. It provides an integrated environment for circuit design, simulation, and analysis using tools such as KiCad, Ngspice, GHDL, and Verilator through a Python-based graphical interface.

This internship project focuses on the integration of **OpenROAD**, an open-source RTL-to-GDSII digital physical design flow, with eSim. The objective of the project was to enable users to directly perform backend physical design operations from the eSim environment, thereby extending eSim from circuit simulation to complete digital layout generation.

The implementation involved modifying the eSim frontend by updating `Application.py` to include an OpenROAD execution button within the graphical user interface. A backend automation module, `OpenROAD.py`, was developed to manage the OpenROAD Flow Scripts (ORFS) execution process. Additionally, a Python-based converter script, `netlist2rtl.py`, was implemented to automatically generate synthesizable Verilog code from eSim-generated `.cir.out` netlist files.

The integrated workflow was validated using two example circuits: **FullAdder** and **HalfAdder**. The generated Verilog files were successfully processed through the OpenROAD RTL-to-GDSII flow, producing backend design outputs including synthesized Verilog, DEF floorplan files, GDSII layout files, timing constraint files, and parasitic extraction reports.

The generated layouts and backend outputs were verified using the OpenROAD GUI and KLayout visualization tools. Multiple debugging and integration issues related to file handling, OpenROAD execution paths, DEF loading, RTL generation, and GUI execution were identified and resolved during the implementation process.

The complete implementation and project repositories are publicly available at:

- https://github.com/FOSSEE/eSim-to-OpenROAD_Design_Flow_Plugin
- <https://github.com/RISHABH12005/eSim-OpenROAD-Plugin>

Contents

Acknowledgement	1
Abstract	2
1 Introduction to eSim	7
1.1 Overview	7
1.2 Open-Source Circuit Simulation	7
1.3 eSim Workspace and Project Structure	7
1.4 Project Organisation	8
1.5 Internship Objective	9
1.6 Project Directory Structure	9
2 eSim Installation and Setup	11
2.1 System Requirements	11
2.2 Standard eSim Installation	11
2.3 Required Dependencies	12
2.4 Workspace Setup and Project Creation	12
2.5 GUI Overview and OpenROAD Button	12
2.6 Installation Verification	13
3 OpenROAD Project and Installation	15
3.1 OpenROAD Overview	15
3.2 OpenROAD Flow Scripts (ORFS)	15
3.3 OpenROAD GUI	16
3.4 OpenROAD Installation	16
3.5 Yosys and KLayout Setup	17
3.6 Environment Variables and Path Configuration	17
3.7 ORFS Directory Structure	17
4 OpenROAD Integration with eSim	19
4.1 Architecture Overview	19

4.2	Modification of Application.py	19
4.3	OpenROAD Button Handler	20
4.4	Implementation of netlist2rtl.py	21
4.4.1	Netlist Parsing	22
4.4.2	Verilog Generation	23
4.5	Implementation of OpenROAD.py	23
4.5.1	config.mk Generation	24
4.5.2	SDC File Generation	24
4.6	ORFS Execution	25
4.7	Validation of .cir.out Files	26
4.8	Debugging and Fixes	26
4.8.1	ORFS Path Errors	26
4.8.2	HalfAdder Verilog Generation Issue	26
4.8.3	DEF Loading Error in OpenROAD GUI	27
4.9	FullAdder and HalfAdder Validation	27
5	Example Circuit and Testing	30
5.1	Overview	30
5.2	FullAdder Testing	30
5.2.1	Input: FullAdder.cir.out	30
5.2.2	Generated Verilog	30
5.2.3	Generated SDC File	31
5.2.4	Flow Completion	32
5.2.5	Generated Output Files	32
5.3	OpenROAD GUI Visualization	33
5.4	KLayout Visualization	33
5.5	HalfAdder Testing	34
5.6	Workflow Summary	35
6	Future Enhancement and Left Off	36
6.1	Current Status	36
6.2	Completed Work	36
6.3	Support for Additional Circuit Types	36
6.4	Improved RTL Parsing	37
6.5	Improved GUI Integration	37
6.6	Automatic File Management	38
6.7	Enhanced Layout Visualization	38
7	Conclusion	39

7.1	Summary	39
7.2	Learning Outcomes	39
7.3	Final Remarks	40
	References	41

List of Figures

1.1	eSim workspace file manager showing project structure.	8
1.2	FullAdder project structure inside the eSim project panel.	9
2.1	OpenROAD integration button added to the eSim GUI toolbar. . . .	13
2.2	Terminal output showing the eSim project build and ORFS execution process.	14
3.1	OpenROAD GUI showing the generated physical design layout. . . .	16
3.2	Terminal output during OpenROAD Flow Scripts execution.	18
4.1	OpenROAD button integrated into the eSim GUI toolbar.	21
4.2	FullAdder and HalfAdder projects inside the eSim workspace.	28
4.3	FullAdder project during OpenROAD execution.	29
5.1	Successful RTL-to-GDSII flow completion for the FullAdder circuit. .	32
5.2	FullAdder layout visualized in the OpenROAD GUI.	33
5.3	FullAdder GDS layout visualized in KLayout.	34
5.4	FullAdder and HalfAdder after successful RTL-to-GDSII flow execution.	35
7.1	Successful RTL-to-GDSII flow completion for the FullAdder circuit. .	40

Chapter 1

Introduction to eSim

1.1 Overview

eSim (previously known as Oscad/FreeEDA) is a free and open-source Electronic Design Automation (EDA) tool developed by the FOSSEE (Free/Libre and Open Source Software for Education) project at IIT Bombay. It provides an integrated environment for circuit design, simulation, analysis, and PCB design using open-source tools such as KiCad, Ngspice, GHDL, and Verilator through a unified Python-based graphical user interface built using Python and PyQt5.

eSim serves as an accessible alternative to proprietary EDA software such as OrCAD, PSpice, and LTspice. Its open-source architecture allows students, researchers, and developers to study the internal implementation of the tool, contribute improvements, and extend functionalities according to project requirements.

The platform is supported under the National Mission on Education through ICT (NMEICT), Ministry of Education, Government of India, with the objective of promoting open-source software usage in engineering education and research.

1.2 Open-Source Circuit Simulation

The primary simulation engine used in eSim is Ngspice, which supports DC, AC, transient, and noise analysis. KiCad is integrated for schematic capture, while GHDL and Verilator enable mixed-signal simulation support through the NGHDL subsystem.

The modular and open-source nature of eSim makes it highly suitable for educational projects, internship contributions, and research-oriented development. During this internship, the same architecture was extended to integrate OpenROAD for RTL-to-GDSII backend physical design flow support.

1.3 eSim Workspace and Project Structure

The eSim workspace organizes individual circuits as separate project directories. Each project typically contains simulation files, generated netlists, schematic files, and backend outputs required during circuit analysis and implementation.

Common project files include:

- `.cir` — KiCad-generated circuit netlist
- `.cir.out` — processed Ngspice-compatible netlist
- `.sch` — schematic design file
- `.sub` — subcircuit definitions

Figure 1.1 shows the eSim workspace and file manager interface used during the project implementation.

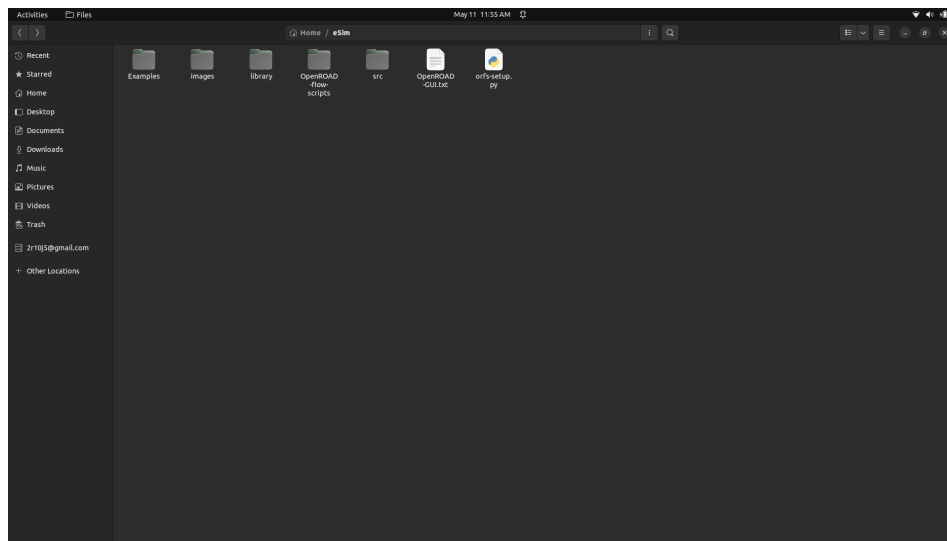


Figure 1.1. eSim workspace file manager showing project structure.

1.4 Project Organisation

The OpenROAD integration workflow was tested using example circuit projects such as **FullAdder** and **HalfAdder**. Each project directory contained generated netlists, synthesized Verilog files, backend physical design outputs, and generated layout files.

Figure 1.2 shows the FullAdder project structure inside the eSim project panel.

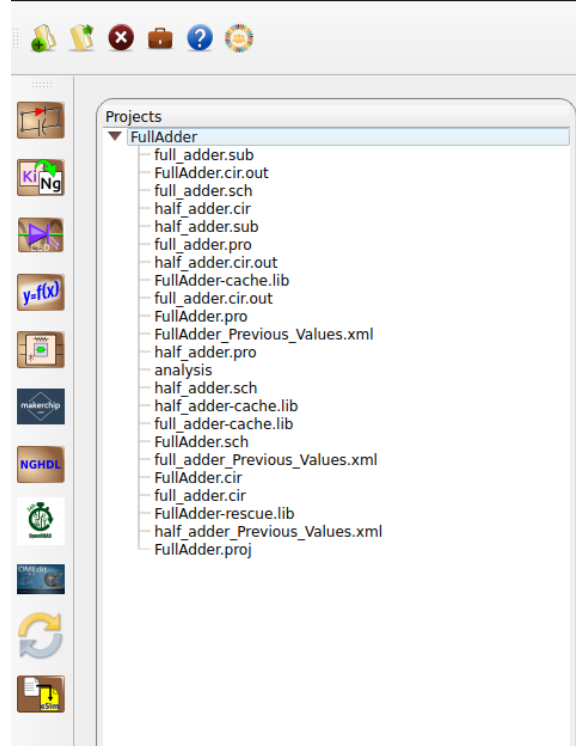


Figure 1.2. FullAdder project structure inside the eSim project panel.

1.5 Internship Objective

The primary objective of this internship was to integrate the **OpenROAD** digital physical design flow into eSim, enabling users to execute a complete RTL-to-GDSII workflow directly from the eSim graphical interface.

The major objectives of the project included:

- Understanding the eSim frontend and project workflow.
- Adding an OpenROAD execution button in the eSim graphical interface.
- Developing a backend Python module to execute OpenROAD Flow Scripts (ORFS).
- Implementing a netlist-to-RTL converter for automatic Verilog generation.
- Generating backend physical design outputs such as DEF and GDSII files.
- Testing the complete workflow using FullAdder and HalfAdder circuits.
- Visualizing generated layouts using OpenROAD GUI and KLayout.

1.6 Project Directory Structure

The integrated OpenROAD-eSim system follows the directory structure shown below:

```

1 eSim/
2 |-- src/
3 |   |-- frontEnd/
4 |   |   '-- Application.py
5 |   |
6 |   '-- maker/
7 |       |-- OpenROAD.py
8 |       '-- netlist2rtl.py
9 |
10 |-- Examples/
11 |   |-- FullAdder/
12 |   |   |-- FullAdder.cir.out
13 |   |   |-- FullAdder.v
14 |   |   |-- FullAdder.def
15 |   |   |-- FullAdder.gds
16 |   |   '-- logs/
17 |   |
18 |   '-- HalfAdder/
19 |       |-- HalfAdder.cir.out
20 |       |-- HalfAdder.v
21 |       |-- HalfAdder.def
22 |       |-- HalfAdder.gds
23 |       '-- logs/
24 |
25 '-- OpenROAD-flow-scripts/
26     '-- flow/
27         |-- designs/
28         |   '-- sky130hd/
29         |       |-- FullAdder/
30         |       '-- HalfAdder/
31         |
32         '-- results/
33         '-- sky130hd/
34             |-- FullAdder/
35             '-- HalfAdder/

```

Listing 1.1. Project directory structure used during integration.

The above structure was used throughout the internship for OpenROAD integration, backend automation, testing, and generated file management.

Chapter 2

eSim Installation and Setup

2.1 System Requirements

Development and testing for the OpenROAD integration were performed on **Ubuntu 22.04 LTS**. The following system configuration was used during implementation and testing:

- **Operating System:** Ubuntu 22.04 LTS
- **RAM:** Minimum 4 GB (8 GB recommended)
- **Storage:** At least 20 GB free disk space
- **Internet Connection:** Required for repository cloning and dependency installation
- **Python Version:** Python 3.8 or later with PyQt5 support

The installation environment also included OpenROAD-flow-scripts (ORFS), KLayout, Yosys, and additional Python dependencies required for backend automation.

2.2 Standard eSim Installation

The eSim source code was cloned from the official FOSSEE GitHub repository and installed using the provided installation script.

```
1 $ git clone https://github.com/FOSSEE/eSim.git
2 $ cd eSim
3 $ chmod +x install-eSim.sh
4 $ ./install-eSim.sh --install
```

Listing 2.1. Standard eSim installation commands.

The installation script automatically installs the required packages including Python, PyQt5, Ngspice, KiCad, and related system dependencies.

2.3 Required Dependencies

Additional dependencies were installed to support OpenROAD integration and RTL-to-GDSII backend flow execution.

```
1 $ sudo apt update
2
3 $ sudo apt install -y build-essential git cmake
4 $ sudo apt install -y python3-dev python3-pip
5 $ sudo apt install -y klayout
6
7 $ pip3 install --user pyqt5
```

Listing 2.2. Additional dependencies for OpenROAD integration.

These dependencies were required for GUI execution, Python backend scripting, layout visualization, and ORFS workflow execution.

2.4 Workspace Setup and Project Creation

After installation, eSim was launched from the terminal or application menu. New projects were created using the *New Project* option available in the eSim graphical interface.

Each project generated its own directory containing schematic files, generated netlists, simulation outputs, and backend design files required during OpenROAD execution.

2.5 GUI Overview and OpenROAD Button

The eSim graphical interface provides project management, schematic editing, simulation control, and tool integration support. During this internship, an additional **OpenROAD** execution button was integrated into the main toolbar of the application.

The button was connected with the backend automation flow implemented in `OpenROAD.py`. When triggered, it automatically processed the selected project and initiated the OpenROAD RTL-to-GDSII flow.

Figure 2.1 shows the OpenROAD integration button added to the eSim GUI toolbar.

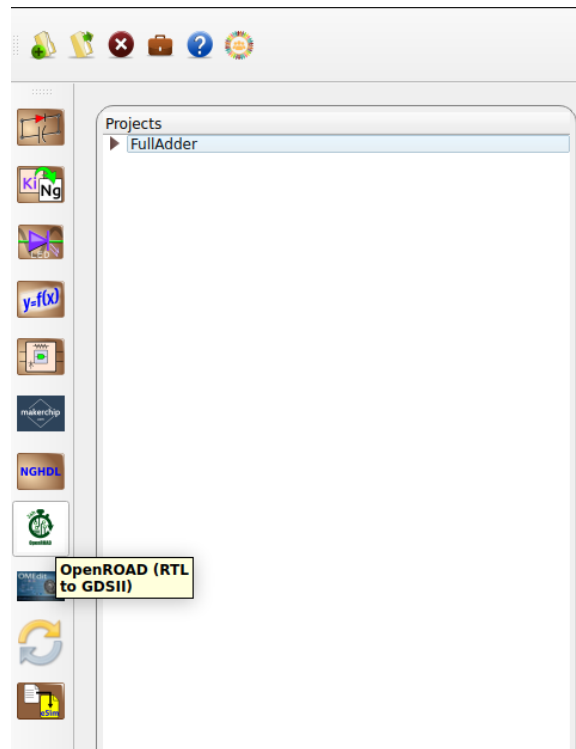


Figure 2.1. OpenROAD integration button added to the eSim GUI toolbar.

2.6 Installation Verification

After installation and integration, the setup was verified by launching eSim and confirming that:

- the graphical interface opened correctly,
- the OpenROAD button was visible in the toolbar,
- existing projects could be loaded successfully,
- generated `.cir.out` files were valid,
- OpenROAD Flow Scripts executed without errors.

Figure 2.2 shows the terminal output generated during project execution and ORFS backend flow processing.

```

/home/rj/eSim/OpenROAD-flow-scripts/flow/scripts/klayout.sh -zz -rd design_name=full_adder \
-rd in_def=./results/sky130hd/full_adder/base/6_final.def \
-rd in_files="/home/rj/eSim/OpenROAD-flow-scripts/flow/platforms/sky130hd/gds/sky130_fd_sc_hd.gds " \
-rd seal_file="" \
-rd out_file=./results/sky130hd/full_adder/base/6_1_merged.gds \
-rd tech_file=./objects/sky130hd/full_adder/base/klayout.lyt \
-rd layer_map= \
-r /home/rj/eSim/OpenROAD-flow-scripts/flow/util/def2stream.py
KLayout 0.30.7
[INFO] Reporting cells prior to loading DEF ...
/home/rj/eSim/OpenROAD-flow-scripts/flow/platforms/sky130hd/gds/sky130_fd_sc_hd.gds
[INFO] All LEF cells have matching GDS/OAS cells
[INFO] No orphan cells in the final layout
Elapsed time: 0:01.31[h:]min:sec. CPU time: user 1.18 sys 0.13 (99%). Peak memory: 441484KB.
cp results/sky130hd/full_adder/base/6_1_merged.gds results/sky130hd/full_adder/base/6_final.gds
./logs/sky130hd/full_adder/base
Log Elapsed/s Peak Memory/MB sha1sum .odb [0:20)
1_1_yosys_canonicalize 0 55 2e8e27d23244d758237c
1_2_yosys 0 57 174a50cf5924490ea536
1_synth 0 110 8e035147b5f21d342015
2_1_floorplan 0 130 b58bb1b67cf765ebba56
2_2_floorplan_macro 0 108 b58bb1b67cf765ebba56
2_3_floorplan_tapcell 0 108 788b1d137ecbc29a7627
2_4_floorplan_pdn 0 111 3052e0d94b0e63fa74d5
3_1_place_gp_skip_io 0 109 af159a517757f9dfa9e5
3_2_place_iop 0 109 7e275682e72a7cdcd2dc
3_3_place_gp 0 208 e3da76bb3e643958daf4
3_4_place_resized 0 129 e3da76bb3e643958daf4
3_5_place_dp 0 112 44e373203591a229e298
4_1_cts 1 137 5cacfa68e546dbacdd56
5_1_grt 0 240 fa2fb80a4eef1907f3bc
5_2_route 0 592 513ca4f7291f9d0eb277
5_3_fillcell 0 113 b2b1905d0440150b29b3
5_1_fill 0 109 b2b1905d0440150b29b3
5_1_merge 1 431 N/A
5_report 1 215 N/A
Total 3 592
[7] Collecting Outputs
Copied : /home/rj/eSim/Examples/FullAdder/full_adder.gds
Copied : /home/rj/eSim/Examples/FullAdder/full_adder.def
Copied : /home/rj/eSim/Examples/FullAdder/full_adder.v
Copied : /home/rj/eSim/Examples/FullAdder/full_adder.sdc
Copied : /home/rj/eSim/Examples/FullAdder/full_adder.spef
Copied Logs : /home/rj/eSim/Examples/FullAdder/logs
Copied Reports : /home/rj/eSim/Examples/FullAdder/reports

===== FLOW COMPLETED =====

Project Directory:
/home/rj/eSim/Examples/FullAdder

[4] Opening GDS in KLayout

```

Figure 2.2. Terminal output showing the eSim project build and ORFS execution process.

Chapter 3

OpenROAD Project and Installation

3.1 OpenROAD Overview

OpenROAD is an open-source RTL-to-GDSII digital physical design flow developed under the DARPA OpenROAD project. It provides an automated backend implementation flow starting from synthesized RTL and ending with a manufacturable GDSII layout.

The OpenROAD ecosystem consists of multiple open-source tools integrated into a unified backend flow. Major components used during this internship include:

- **Yosys** — RTL synthesis tool
- **OpenROAD** — Physical design engine
- **OpenSTA** — Static timing analysis
- **TritonRoute** — Detailed routing engine
- **KLayout** — GDSII layout viewer

The backend implementation flow used during this project was based on the **sky130hd** standard cell library provided by the SkyWater 130 nm open-source PDK.

3.2 OpenROAD Flow Scripts (ORFS)

OpenROAD Flow Scripts (ORFS) provide an automated Makefile-based backend implementation flow for RTL-to-GDSII design generation.

The complete flow includes the following stages:

1. **Synthesis** — Verilog RTL is synthesized into a gate-level netlist using Yosys.
2. **Floorplanning** — Core area and die area are generated for physical implementation.

3. **Placement** — Standard cells are placed inside the core region.
4. **Clock Tree Synthesis (CTS)** — Clock distribution buffers are inserted.
5. **Routing** — Global routing and detailed routing are performed.
6. **GDSII Generation** — Final layout is exported as a `.gds` file.

The ORFS infrastructure was used as the primary backend flow engine during the OpenROAD integration with eSim.

3.3 OpenROAD GUI

The OpenROAD GUI provides a graphical environment for viewing floorplans, standard-cell placement, routing information, and final layout structures generated during backend implementation.

Figure 3.1 shows the OpenROAD GUI displaying the generated backend layout for the FullAdder design.

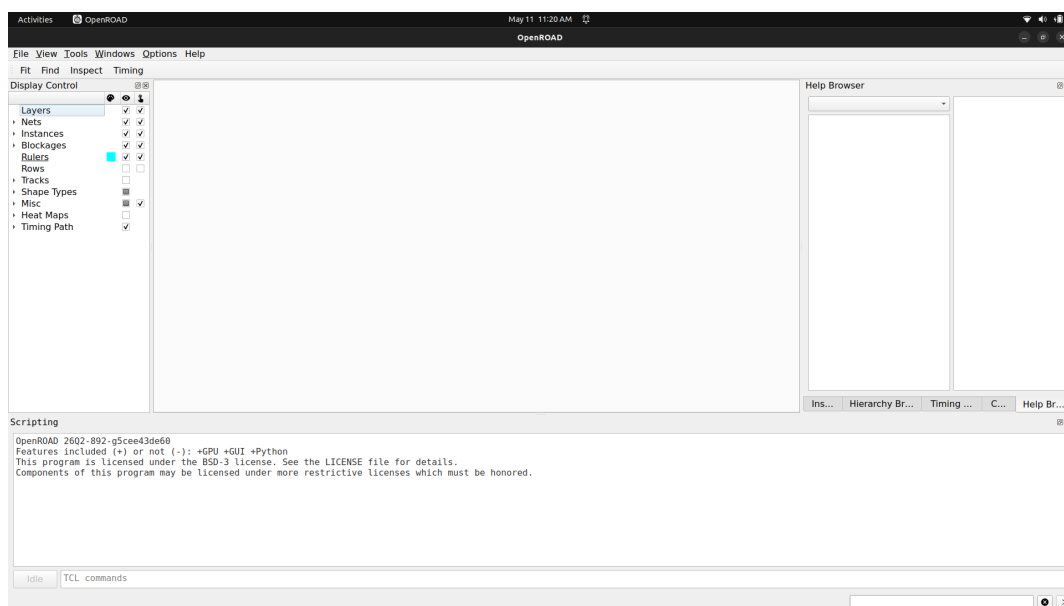


Figure 3.1. OpenROAD GUI showing the generated physical design layout.

3.4 OpenROAD Installation

OpenROAD Flow Scripts were installed by cloning the official repository and executing the provided setup scripts.

```

1 $ git clone --recursive \
2 https://github.com/The-OpenROAD-Project/OpenROAD-flow-scripts
3
4 $ cd OpenROAD-flow-scripts
5
6 $ sudo ./setup.sh

```

```

7
8 $ ./build_openroad.sh --local

```

Listing 3.1. OpenROAD Flow Scripts installation commands.

The installation process downloads required dependencies, builds OpenROAD from source, and installs the required backend tools locally inside the ORFS directory.

After compilation, the OpenROAD executable becomes available at:

```

1 OpenROAD-flow-scripts/tools/install/OpenROAD/bin/openroad

```

3.5 Yosys and KLayout Setup

Yosys was installed as part of the OpenROAD Flow Scripts build process. KLayout was installed separately for GDSII layout visualization and inspection.

```

1 $ sudo apt install -y klayout

```

Listing 3.2. KLayout installation command.

The generated layout files were verified using both OpenROAD GUI and KLayout during backend testing.

3.6 Environment Variables and Path Configuration

The backend integration module `OpenROAD.py` was configured to automatically locate the OpenROAD and Yosys binaries inside the ORFS installation directory.

The following configuration paths were used:

```

1 ORFS_PATH = "/home/rj/eSim/OpenROAD-flow-scripts"
2
3 OPENROAD_BIN = ORFS_PATH + \
4 "/tools/install/OpenROAD/bin/openroad"
5
6 YOSYS_BIN = ORFS_PATH + \
7 "/tools/install/yosys/bin/yosys"

```

Listing 3.3. Tool path configuration inside `OpenROAD.py`.

These paths were configured directly inside the backend module to avoid manual environment variable configuration during execution.

3.7 ORFS Directory Structure

The OpenROAD Flow Scripts infrastructure follows a structured design hierarchy for storing flow configuration files and generated backend outputs.

```

1 OpenROAD-flow-scripts/
2 '-- flow/
3   |-- designs/

```

```

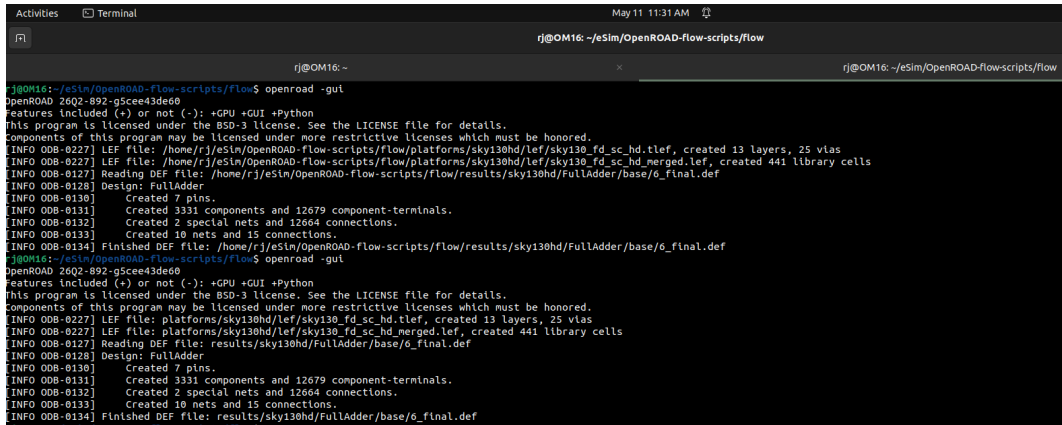
4 | ' -- sky130hd/
5 | | -- FullAdder/
6 | | | -- config.mk
7 | | | -- constraint.sdc
8 | |
9 | | -- HalfAdder/
10 | | | -- config.mk
11 | | | -- constraint.sdc
12 |
13 | -- results/
14 | -- sky130hd/
15 | | -- FullAdder/
16 | | | -- 6_final.gds
17 | | | -- 6_final.def
18 | | | -- ...
19 | |
20 | -- HalfAdder/
21 | | -- ...

```

Listing 3.4. ORFS design directory structure.

The **designs/** directory stores configuration and constraint files, while the **results/** directory stores generated backend outputs such as DEF and GDSII layouts.

Figure 3.2 shows the terminal output generated during OpenROAD Flow Scripts execution.



```

Activities  Terminal  May 11 11:31 AM
rj@OM16: ~/eSim/OpenROAD-flow-scripts/flow
rj@OM16: ~/eSim/OpenROAD-flow-scripts/flow
rj@OM16: ~/eSim/OpenROAD-flow-scripts/flow$ openroad -gui
OpenROAD 2602-892-g5ce43de68
Features included (+) or not (-): +GPU +GUI +Python
This program is licensed under the BSD-3 license. See the LICENSE file for details.
Components of this program may be licensed under more restrictive licenses which must be honored.
[INFO ODB-0227] LEF file: /home/rj/eSim/OpenROAD-flow-scripts/flow/platforms/sky130hd/lef/sky130_fd_sc_hd.tlef, created 13 layers, 25 vias
[INFO ODB-0227] LEF file: /home/rj/eSim/OpenROAD-flow-scripts/flow/platforms/sky130hd/lef/sky130_fd_sc_hd_merged.tlef, created 441 library cells
[INFO ODB-0127] Reading DEF file: /home/rj/eSim/OpenROAD-flow-scripts/flow/results/sky130hd/FullAdder/base/6_final.def
[INFO ODB-0128] Design: FullAdder
[INFO ODB-0130] Created 7 pins.
[INFO ODB-0131] Created 3331 components and 12679 component-terminals.
[INFO ODB-0132] Created 2 special nets and 12664 connections.
[INFO ODB-0133] Created 10 nets and 15 connections.
[INFO ODB-0134] Finished DEF file: /home/rj/eSim/OpenROAD-flow-scripts/flow/results/sky130hd/FullAdder/base/6_final.def
rj@OM16: ~/eSim/OpenROAD-flow-scripts/flow$ openroad -gui
OpenROAD 2602-892-g5ce43de68
Features included (+) or not (-): +GPU +GUI +Python
This program is licensed under the BSD-3 license. See the LICENSE file for details.
Components of this program may be licensed under more restrictive licenses which must be honored.
[INFO ODB-0227] LEF file: platforms/sky130hd/lef/sky130_fd_sc_hd.tlef, created 13 layers, 25 vias
[INFO ODB-0227] LEF file: platforms/sky130hd/lef/sky130_fd_sc_hd_merged.tlef, created 441 library cells
[INFO ODB-0127] Reading DEF file: results/sky130hd/FullAdder/base/6_final.def
[INFO ODB-0128] Design: FullAdder
[INFO ODB-0130] Created 7 pins.
[INFO ODB-0131] Created 3331 components and 12679 component-terminals.
[INFO ODB-0132] Created 2 special nets and 12664 connections.
[INFO ODB-0133] Created 10 nets and 15 connections.
[INFO ODB-0134] Finished DEF file: results/sky130hd/FullAdder/base/6_final.def

```

Figure 3.2. Terminal output during OpenROAD Flow Scripts execution.

Chapter 4

OpenROAD Integration with eSim

This chapter describes the complete implementation work carried out during the integration of OpenROAD with eSim. It explains the frontend modifications, backend automation modules, netlist conversion process, OpenROAD Flow Scripts execution, and the debugging steps performed during development.

4.1 Architecture Overview

The integration connects the eSim simulation environment with the OpenROAD RTL-to-GDSII backend flow. The overall workflow implemented during this internship is shown below:

1. The user selects a project inside eSim and clicks the **OpenROAD** button.
2. The button handler inside `Application.py` identifies the selected project and locates the generated `.cir.out` file.
3. The `netlist2rtl.py` module parses the `.cir.out` file and generates a synthesizable Verilog (`.v`) file.
4. The `OpenROAD.py` backend module automatically generates the required ORFS configuration files such as `config.mk` and `constraint.sdc`.
5. OpenROAD Flow Scripts execute synthesis, floorplanning, placement, CTS, routing, and GDSII generation.
6. The generated layout is visualized automatically using the OpenROAD GUI and KLayout.

4.2 Modification of `Application.py`

The eSim frontend is primarily controlled through the file `src/frontEnd/Application.py`. During this internship, a new OpenROAD execution button was added to the left-side toolbar of the graphical interface.

The following code snippet shows the implementation used for adding the OpenROAD button:

```
1 self.openroadAction = QtWidgets.QAction(  
2     QtGui.QIcon(os.path.join(  
3         self.resource_path,  
4         'openroad.png')),  
5     'OpenROAD',  
6     self  
7 )  
8  
9 self.openroadAction.setStatusTip(  
10     'Run OpenROAD Flow'  
11 )  
12  
13 self.openroadAction.triggered.connect(  
14     self.openOpenROAD  
15 )  
16  
17 self.leftToolBar.addAction(  
18     self.openroadAction  
19 )
```

Listing 4.1. Adding the OpenROAD button inside Application.py.

The button was connected with a backend execution handler that validates the selected project and launches the OpenROAD flow.

4.3 OpenROAD Button Handler

The handler function retrieves the currently selected project directory, validates the generated .cir.out file, and initializes the backend execution module.

```
1 def openOpenROAD(self):  
2  
3     project_dir = self.obj_projectExplorer.curProject  
4  
5     if not project_dir:  
6         QtWidgets.QMessageBox.warning(  
7             self,  
8             'No Project',  
9             'Please select a project.'  
10        )  
11        return  
12  
13     project_name = os.path.basename(project_dir)  
14  
15     cir_out = os.path.join(  
16         project_dir,  
17         project_name + '.cir.out'  
18     )  
19  
20     if not os.path.isfile(cir_out):
```

```

21         QtWidgets.QMessageBox.warning(
22             self,
23             'File Not Found',
24             'Run KiCad-to-Ngspice conversion first.'
25         )
26         return
27
28     from maker.OpenROAD import OpenROADFlow
29
30     self.openroad_obj = OpenROADFlow(
31         cir_out,
32         project_dir
33     )
34
35     self.openroad_obj.run()

```

Listing 4.2. OpenROAD execution handler in Application.py.

Figure 4.1 shows the OpenROAD button integrated into the eSim GUI toolbar.

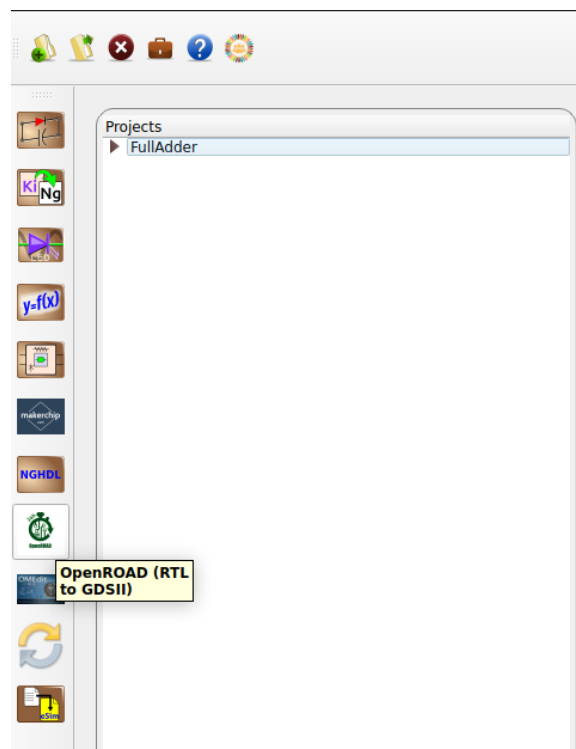


Figure 4.1. OpenROAD button integrated into the eSim GUI toolbar.

4.4 Implementation of netlist2rtl.py

The `netlist2rtl.py` module was developed to automatically convert eSim-generated `.cir.out` files into synthesizable Verilog RTL.

The parser reads the SPICE netlist line by line and extracts:

- module name,

- input ports,
- output ports,
- subcircuit instances,
- internal net connections.

4.4.1 Netlist Parsing

The following logic was used for parsing the `.cir.out` file:

```
1 def parse_cir_out(cir_out_path, project_dir):
2
3     module_name = os.path.basename(project_dir)
4
5     ports_in = []
6     ports_out = []
7     instances = []
8
9     with open(cir_out_path, 'r') as f:
10
11         for line in f:
12
13             line = line.strip()
14
15             if line.startswith('*') or not line:
16                 continue
17
18             if line.lower().startswith('v'):
19
20                 parts = line.split()
21
22                 if len(parts) >= 3:
23
24                     net = parts[1]
25
26                     if net.lower() not in ('gnd', '0'):
27                         ports_in.append(net)
28
29             if line.lower().startswith('x'):
30                 instances.append(line)
31
32     return (
33         module_name,
34         ports_in,
35         ports_out,
36         instances
37     )
```

Listing 4.3. Core parsing logic in `netlist2rtl.py`.

4.4.2 Verilog Generation

After parsing, the extracted information is converted into a Verilog module.

```

1  def generate_verilog(
2      module_name,
3      ports_in,
4      ports_out,
5      instances,
6      output_path
7  ):
8
9      lines = []
10
11     all_ports = ports_in + ports_out
12
13     lines.append(
14         f'module {module_name} ('
15     )
16
17     lines.append(
18         '    ' + ',\n    '.join(all_ports) + ');'
19     )
20
21     for p in ports_in:
22         lines.append(f'input {p};')
23
24     for p in ports_out:
25         lines.append(f'output {p};')
26
27     for inst in instances:
28         lines.append(' ' + inst + ';')
29
30     lines.append('endmodule')
31
32     with open(output_path, 'w') as f:
33         f.write('\n'.join(lines) + '\n')

```

Listing 4.4. Verilog generation in netlist2rtl.py.

4.5 Implementation of OpenROAD.py

The `OpenROAD.py` module serves as the primary backend controller for ORFS execution.

The module performs the following tasks:

- validates input files,
- generates ORFS configuration files,
- creates timing constraints,
- launches ORFS execution,

- verifies generated outputs,
- opens the generated layout in OpenROAD GUI.

4.5.1 config.mk Generation

The following configuration file was generated automatically for each project:

```
1 def generate_config_mk(  
2     design_name,  
3     verilog_path,  
4     output_dir  
5 ):  
6  
7     config = f"""  
8 export DESIGN_NICKNAME = {design_name}  
9 export DESIGN_NAME     = {design_name}  
10 export PLATFORM        = sky130hd  
11  
12 export VERILOG_FILES   = {verilog_path}  
13  
14 export SDC_FILE = \  
15 ./designs/${PLATFORM}/\  
16 ${DESIGN_NICKNAME}/constraint.sdc  
17  
18 export CORE_UTILIZATION = 40  
19 export CORE_ASPECT_RATIO = 1  
20 export CORE_MARGIN      = 2  
21 export PLACE_DENSITY    = 0.65  
22 """  
23  
24     config_path = os.path.join(  
25         output_dir,  
26         'config.mk'  
27     )  
28  
29     with open(config_path, 'w') as f:  
30         f.write(config.strip())  
31  
32     return config_path
```

Listing 4.5. config.mk generation inside OpenROAD.py.

4.5.2 SDC File Generation

Timing constraints were automatically generated using the following logic:

```
1 def generate_sdc(  
2     design_name,  
3     ports_in,  
4     ports_out,
```

```

5     output_path
6 ):
7
8     lines = [
9         f'current_design {design_name}',
10        f'create_clock -name clk -period 10.0000'
11    ]
12
13    for p in ports_in:
14        lines.append(
15            f'set_input_delay 0.1000 ',
16            f'-add_delay [get_ports {{{p}}}]',
17        )
18
19    for p in ports_out:
20        lines.append(
21            f'set_output_delay 0.1000 ',
22            f'-add_delay [get_ports {{{p}}}]',
23        )
24
25    with open(output_path, 'w') as f:
26        f.write('\n'.join(lines) + '\n')

```

Listing 4.6. Automatic SDC generation.

4.6 ORFS Execution

Once configuration files were generated successfully, OpenROAD Flow Scripts were executed using the Makefile-based backend flow.

```

1 def run_orfs(design_name, orfs_path):
2
3     cmd = [
4         'make',
5         f'DESIGN_CONFIG=./designs/sky130hd/'
6         f'{design_name}/config.mk'
7     ]
8
9     result = subprocess.run(
10        cmd,
11        cwd=os.path.join(orfs_path, 'flow'),
12        text=True
13    )
14
15    if result.returncode != 0:
16        raise RuntimeError(
17            f'ORFS failed for {design_name}'
18        )

```

Listing 4.7. ORFS execution from OpenROAD.py.

4.7 Validation of .cir.out Files

Before launching the OpenROAD flow, validation checks were added to verify that the generated .cir.out file existed and contained valid SPICE data.

```

1 def validate_cir_out(cir_out_path):
2
3     if not os.path.isfile(cir_out_path):
4         return False, 'File not found.'
5
6     with open(cir_out_path, 'r') as f:
7         content = f.read().lower()
8
9     if '.end' not in content:
10        return False, 'Invalid .cir.out file.'
11
12    return True, 'OK'

```

Listing 4.8. Validation of .cir.out files.

4.8 Debugging and Fixes

Several issues were encountered during implementation and testing. These issues were identified and resolved during the integration process.

4.8.1 ORFS Path Errors

Incorrect project naming initially caused ORFS configuration paths to fail. The issue was resolved by sanitizing project names before directory creation.

```

1 import re
2
3 design_name = re.sub(
4     r'[^A-Za-z0-9_]',
5     '',
6     project_name.replace(' ', '_')
7 )

```

Listing 4.9. Project name sanitization.

4.8.2 HalfAdder Verilog Generation Issue

The generated Verilog file for the HalfAdder project was initially empty because of case-sensitive module naming differences.

The issue was resolved by extracting the module name directly from the project directory instead of parsing it from the netlist.

```

1 module_name = os.path.basename(project_dir)

```

Listing 4.10. Module name extraction fix.

4.8.3 DEF Loading Error in OpenROAD GUI

Initially, the OpenROAD GUI failed to load the generated DEF file correctly because LEF technology files were not loaded.

The issue was resolved by launching the GUI using the generated .odb database file.

```

1  def launch_openroad_gui(
2      orfs_path,
3      design_name
4  ):
5
6      result_dir = os.path.join(
7          orfs_path,
8          'flow',
9          'results',
10         'sky130hd',
11         design_name
12     )
13
14     odb_file = os.path.join(
15         result_dir,
16         '6_final.odb'
17     )
18
19     cmd = [
20         os.path.join(
21             orfs_path,
22             'tools',
23             'install',
24             'OpenROAD',
25             'bin',
26             'openroad'
27         ),
28         '-gui',
29         odb_file
30     ]
31
32     subprocess.Popen(cmd)

```

Listing 4.11. Corrected OpenROAD GUI launch.

4.9 FullAdder and HalfAdder Validation

The complete integration flow was tested using both **FullAdder** and **HalfAdder** circuits.

The integration successfully generated:

- synthesized Verilog files,
- DEF floorplan files,

- GDSII layout files,
- timing constraint files,
- parasitic extraction outputs.

Figure 4.2 shows both projects visible inside the eSim project panel.

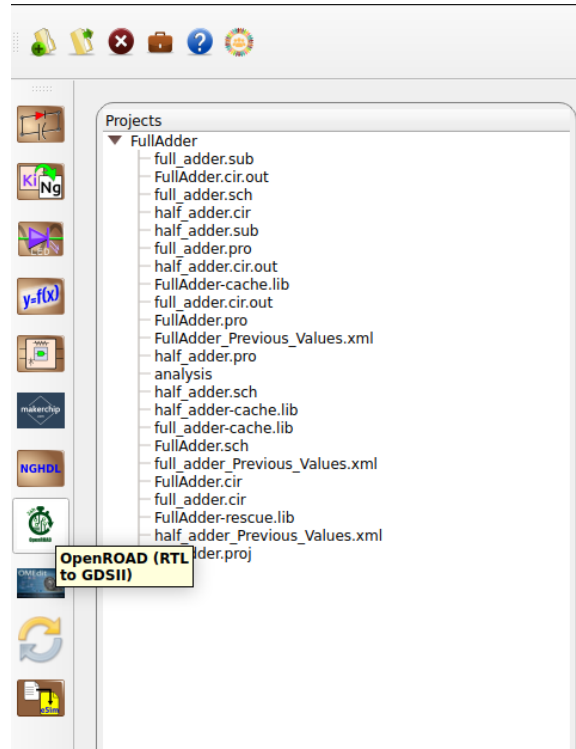


Figure 4.2. FullAdder and HalfAdder projects inside the eSim workspace.

Figure 4.3 shows the FullAdder project during OpenROAD flow execution.

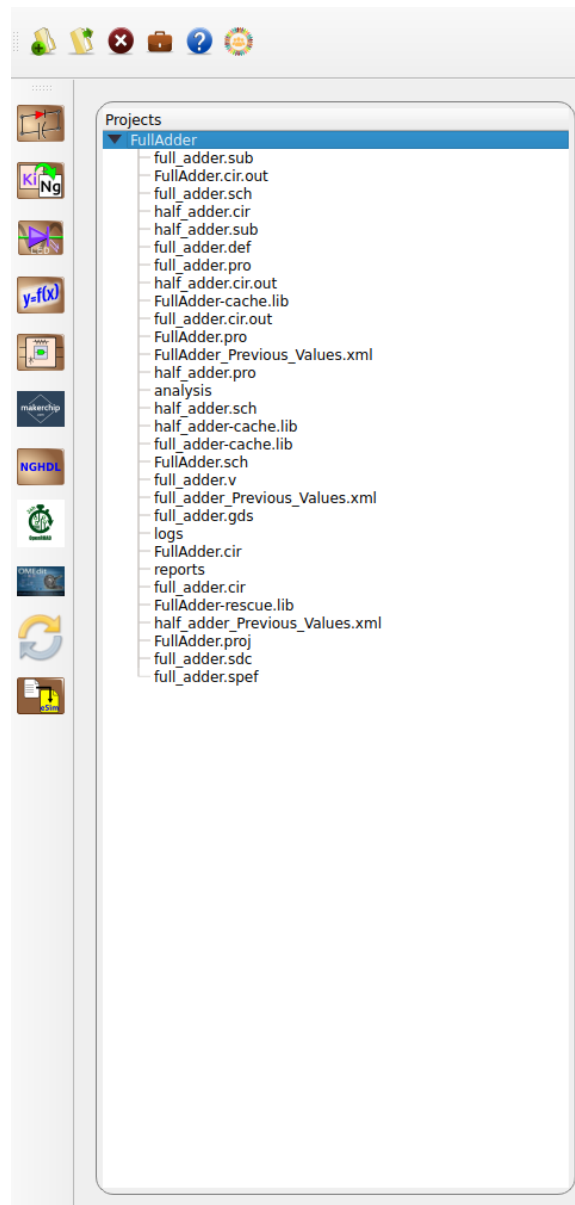


Figure 4.3. FullAdder project during OpenROAD execution.

Chapter 5

Example Circuit and Testing

5.1 Overview

Two example circuits were used to validate the complete OpenROAD integration workflow: **FullAdder** and **HalfAdder**. Both circuits were successfully processed through the complete RTL-to-GDSII backend flow and the generated outputs were verified at each implementation stage.

5.2 FullAdder Testing

5.2.1 Input: FullAdder.cir.out

The FullAdder project uses a `.cir.out` netlist generated from the KiCad schematic inside eSim. The circuit implements a 1-bit FullAdder with three input ports (`in1`, `in2`, and `cin`) and two output ports (`sum` and `cout`).

5.2.2 Generated Verilog

The `netlist2rtl.py` converter processed the generated `FullAdder.cir.out` file and automatically generated the following Verilog module:

```
1  module FullAdder (  
2      cin,  
3      cout,  
4      in1,  
5      in2,  
6      sum  
7  );  
8  
9  input cin;  
10 output cout;  
11 input in1;  
12 input in2;  
13 output sum;  
14
```

```
15 wire net1;
16 wire net2;
17 wire net3;
18 wire net4;
19 wire net5;
20
21 sky130_fd_sc_hd__fa_1 _0_ (
22     .A(net2),
23     .B(net3),
24     .CIN(net1),
25     .COUT(net4),
26     .SUM(net5)
27 );
28
29 sky130_fd_sc_hd__clkdlybuf4s50_1 input1 (
30     .A(cin),
31     .X(net1)
32 );
33
34 sky130_fd_sc_hd__clkdlybuf4s50_1 input2 (
35     .A(in1),
36     .X(net2)
37 );
38
39 sky130_fd_sc_hd__clkdlybuf4s50_1 input3 (
40     .A(in2),
41     .X(net3)
42 );
43
44 sky130_fd_sc_hd__clkdlybuf4s50_1 output4 (
45     .A(net4),
46     .X(cout)
47 );
48
49 sky130_fd_sc_hd__clkdlybuf4s50_1 output5 (
50     .A(net5),
51     .X(sum)
52 );
53
54 endmodule
```

Listing 5.1. Generated FullAdder.v module.

5.2.3 Generated SDC File

The timing constraints file was automatically generated during ORFS setup.

```
1 current_design FullAdder
2
3 create_clock -name clk -period 10.0000
4
5 set_input_delay 0.1000 -add_delay \
```



```

6 [get_ports {cin}]
7
8 set_input_delay 0.1000 -add_delay \
9 [get_ports {in1}]
10
11 set_input_delay 0.1000 -add_delay \
12 [get_ports {in2}]
13
14 set_output_delay 0.1000 -add_delay \
15 [get_ports {cout}]
16
17 set_output_delay 0.1000 -add_delay \
18 [get_ports {sum}]

```

Listing 5.2. Auto-generated FullAdder constraint.sdc file.

5.2.4 Flow Completion

The OpenROAD Flow Scripts execution for the FullAdder circuit completed successfully through all backend stages including synthesis, floorplanning, placement, CTS, routing, and GDSII generation.

Figure 5.1 shows the terminal output after successful completion of the RTL-to-GDSII flow.

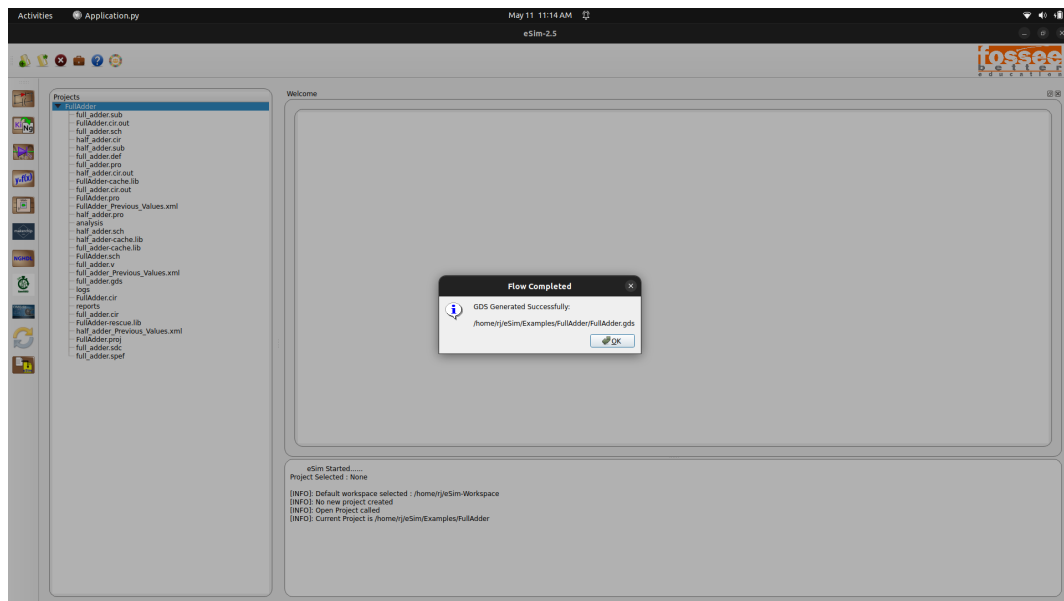


Figure 5.1. Successful RTL-to-GDSII flow completion for the FullAdder circuit.

5.2.5 Generated Output Files

After successful ORFS execution, multiple backend design files were generated automatically.

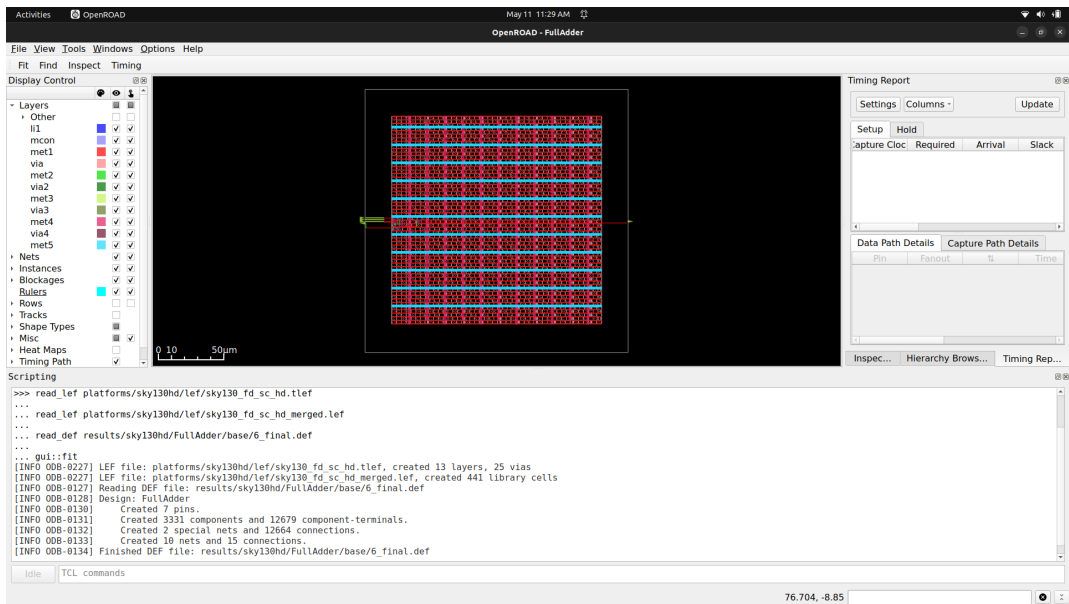
Table 5.1. Generated output files after ORFS execution.

File	Format	Description
FullAdder.v	Verilog	Generated RTL module
FullAdder.def	DEF	Placement and routing data
FullAdder.gds	GDSII	Final layout file
FullAdder.sdc	SDC	Timing constraints
FullAdder.spef	SPEF	Parasitic extraction output
logs/	Directory	Backend execution logs

5.3 OpenROAD GUI Visualization

After completion of the backend flow, the generated layout was opened automatically in the OpenROAD GUI using the generated .odb database file.

Figure 5.2 shows the FullAdder physical layout displayed in the OpenROAD GUI.

**Figure 5.2.** FullAdder layout visualized in the OpenROAD GUI.

5.4 KLayout Visualization

The generated GDSII layout file was also verified using KLayout.

Figure 5.3 shows the FullAdder layout loaded in KLayout for physical inspection.

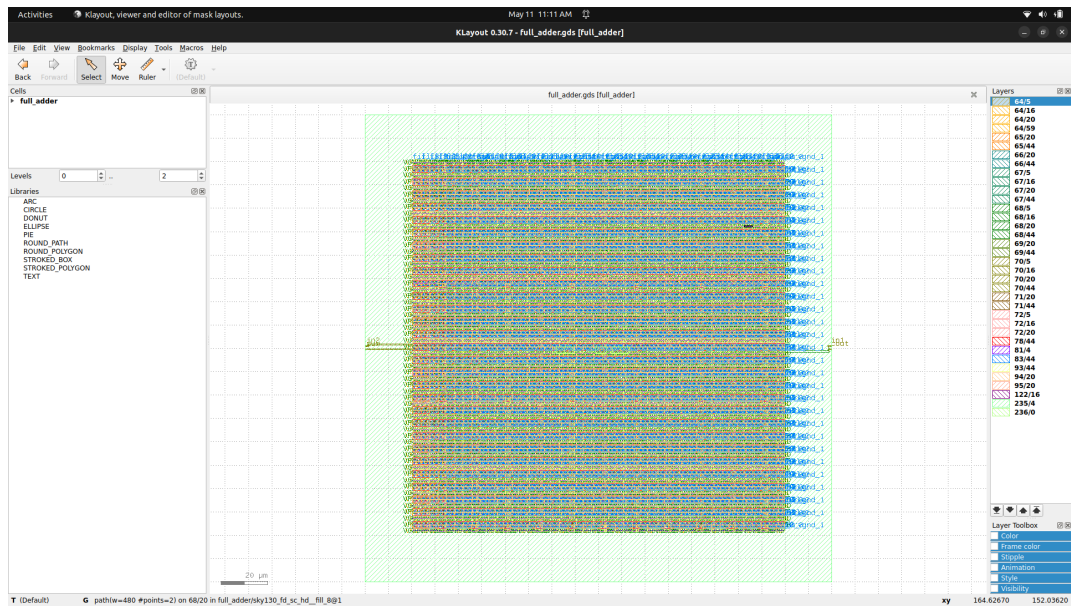


Figure 5.3. FullAdder GDS layout visualized in KLayout.

5.5 HalfAdder Testing

The HalfAdder circuit was also validated successfully using the same OpenROAD integration workflow. The generated `HalfAdder.cir.out` file was converted into Verilog using `netlist2rtl.py`, and the OpenROAD Flow Scripts execution completed successfully.

The generated backend outputs included:

- synthesized Verilog,
- DEF layout files,
- GDSII layout files,
- timing constraint files,
- ORFS execution logs.

Figure 5.4 shows both FullAdder and HalfAdder projects after successful flow completion.

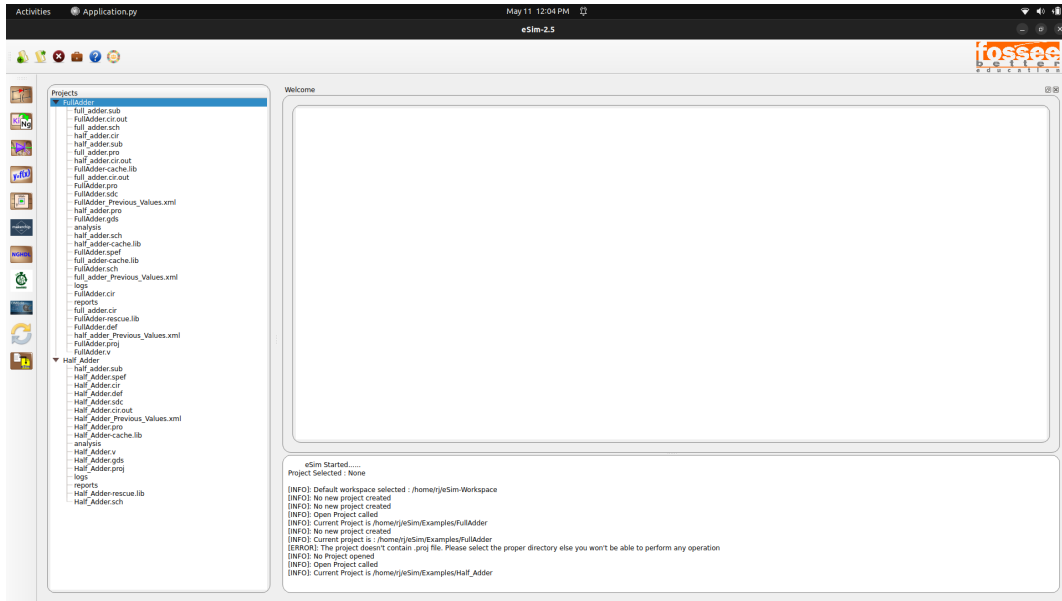


Figure 5.4. FullAdder and HalfAdder after successful RTL-to-GDSII flow execution.

5.6 Workflow Summary

The complete validated workflow implemented during the internship is summarized below:

1. Create and simulate the circuit in eSim using KiCad and Ngspice.
2. Run KiCad-to-Ngspice conversion to generate the `.cir.out` file.
3. Click the OpenROAD button integrated into the eSim toolbar.
4. Automatically generate Verilog RTL using `netlist2rtl.py`.
5. Automatically generate `config.mk` and `constraint.sdc`.
6. Execute the ORFS backend implementation flow.
7. Generate backend outputs including DEF and GDSII layouts.
8. Visualize the generated layouts using OpenROAD GUI and KLayout.

Chapter 6

Future Enhancement and Left Off

6.1 Current Status

The OpenROAD integration with eSim was successfully completed and validated using the **FullAdder** and **HalfAdder** example circuits. The implemented workflow successfully performs automatic Verilog generation, OpenROAD Flow Scripts execution, backend file generation, and layout visualization using both OpenROAD GUI and KLayout.

The RTL-to-GDSII flow was tested end-to-end and the generated backend outputs were verified successfully during implementation.

6.2 Completed Work

At the end of this internship, the following implementation tasks were completed successfully:

- OpenROAD execution button added to the eSim graphical interface.
- `netlist2rtl.py` implemented for automatic Verilog generation.
- `OpenROAD.py` backend module developed for ORFS execution.
- Automatic generation of `config.mk` and `constraint.sdc`.
- Successful RTL-to-GDSII flow execution for FullAdder and HalfAdder.
- Automatic OpenROAD GUI launch after flow completion.
- Successful GDSII visualization using KLayout.
- Multiple backend integration and execution issues resolved.

6.3 Support for Additional Circuit Types

The current implementation primarily supports combinational circuits such as FullAdder and HalfAdder. Future work can extend the parser to support:

- sequential circuits,
- flip-flop based designs,
- hierarchical netlists,
- larger digital modules.

Additional improvements may also be required for handling more complex SPICE netlists generated from larger projects.

6.4 Improved RTL Parsing

The current `netlist2rtl.py` module uses a simplified parsing approach for extracting ports and subcircuit instances from the generated `.cir.out` files.

Future improvements may include:

- improved port identification,
- better wire extraction,
- support for complex net connections,
- automatic detection of input and output signals.

A more robust parser would improve compatibility with larger and more complex circuit designs.

6.5 Improved GUI Integration

The current OpenROAD flow executes as a backend subprocess launched directly from eSim.

Future improvements may include:

- real-time progress display,
- flow execution logs inside the GUI,
- flow cancellation support,
- automatic error reporting,
- generated file summaries after execution.

These features would improve the overall usability of the integration.

6.6 Automatic File Management

Currently, generated backend outputs such as `.def`, `.gds`, and `.spef` files are stored inside the ORFS results directory.

Future improvements may include:

- automatic copying of generated files into the eSim project directory,
- automatic workspace refresh,
- organized output management for multiple projects.

This would simplify project management and improve integration consistency.

6.7 Enhanced Layout Visualization

The generated layouts are currently visualized using external OpenROAD GUI and KLayout applications.

Future improvements may include direct integration of layout visualization inside the eSim graphical interface, allowing users to inspect generated layouts without leaving the eSim environment.

Such integration would provide a more unified open-source RTL-to-GDSII workflow within a single application.

Chapter 7

Conclusion

7.1 Summary

This internship successfully implemented the integration of the **OpenROAD** digital physical design flow into the **eSim** open-source EDA environment. The completed workflow connects eSim circuit simulation with the OpenROAD RTL-to-GDSII backend implementation flow entirely within an open-source ecosystem.

The major implementation tasks completed during this internship include:

1. Integration of an OpenROAD execution button into the eSim graphical interface through modifications in `Application.py`.
2. Development of `netlist2rtl.py` for automatic Verilog generation from eSim-generated `.cir.out` netlists.
3. Development of `OpenROAD.py` for OpenROAD Flow Scripts (ORFS) execution and backend automation.
4. Automatic generation of `config.mk` and `constraint.sdc` files required for ORFS execution.
5. Successful RTL-to-GDSII flow validation for both FullAdder and HalfAdder circuits.
6. Generation of backend outputs including Verilog, DEF, GDSII, SDC, and SPEF files.
7. Successful layout visualization using OpenROAD GUI and KLayout.
8. Identification and resolution of multiple backend integration and execution issues during implementation.

7.2 Learning Outcomes

This internship provided practical exposure to:

- eSim frontend and backend architecture,
- Python and PyQt5 GUI development,
- OpenROAD Flow Scripts (ORFS),
- RTL synthesis using Yosys,
- physical design flow execution,
- GDSII layout visualization using KLayout,
- debugging and integration of open-source EDA tools.

The project also provided hands-on experience with backend automation, flow execution management, and open-source digital design workflows.

7.3 Final Remarks

The completed integration demonstrates that a complete open-source RTL-to-GDSII workflow can be executed directly from the eSim environment. The successful implementation of FullAdder and HalfAdder backend flows confirms that OpenROAD can be effectively integrated into eSim for digital physical design automation.

This integration extends the capabilities of eSim beyond circuit simulation by enabling backend physical design generation within a unified open-source environment.

Figure 7.1 shows the successful RTL-to-GDSII flow completion for the FullAdder circuit.

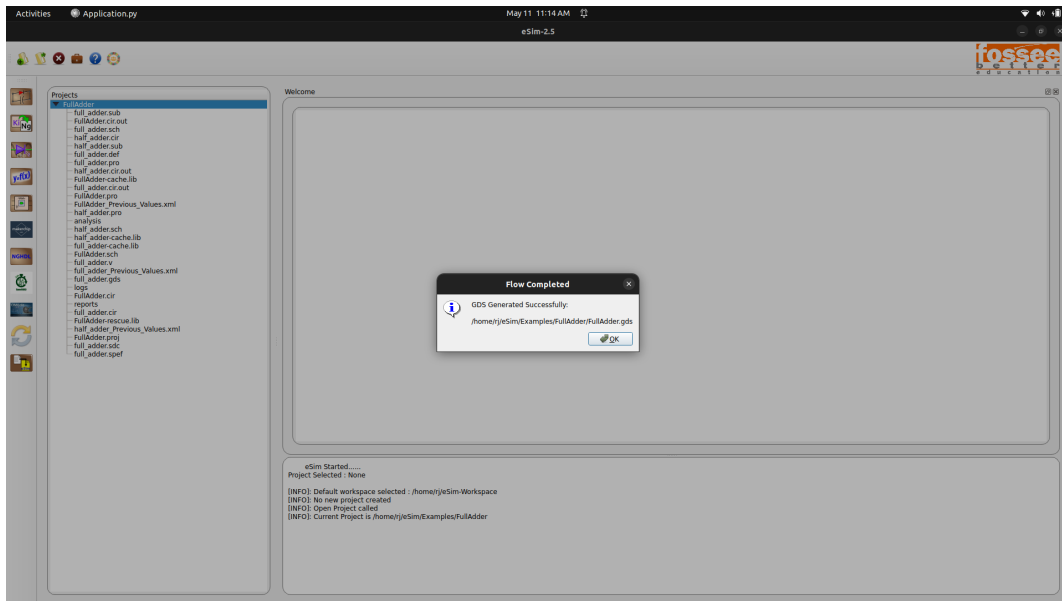


Figure 7.1. Successful RTL-to-GDSII flow completion for the FullAdder circuit.

References

1. FOSSEE, *eSim Repository*, GitHub Repository.
<https://github.com/FOSSEE/eSim>
2. The OpenROAD Project, *OpenROAD-flow-scripts*, GitHub Repository.
<https://github.com/The-OpenROAD-Project/OpenROAD-flow-scripts>
3. The OpenROAD Project, *OpenROAD Repository*, GitHub Repository.
<https://github.com/The-OpenROAD-Project/OpenROAD>
4. The OpenROAD Project, *OpenROAD Official Website*.
<https://theopenroadproject.org/>
5. The OpenROAD Project, *OpenROAD Documentation*.
<https://openroad.readthedocs.io/en/latest/>
6. FOSSEE, *eSim Official Website*.
<https://esim.fossee.in/home>
7. FOSSEE, *eSim-to-OpenROAD Design Flow Plugin*, GitHub Repository.
https://github.com/FOSSEE/eSim-to-OpenROAD_Design_Flow_Plugin
8. Rishabh Jain, *eSim OpenROAD Plugin*, GitHub Repository.
<https://github.com/RISHABH12005/eSim-OpenROAD-Plugin>
9. Rishabh Jain, *eSim Repository Fork*, GitHub Repository.
<https://github.com/RISHABH12005/eSim>